

UTS
PENGOLAHAN CITRA DIGITAL



NAMA : Anggia Putri Tesalonika Pane

NIM : 202431162

KELAS : G

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Muhammad Caisar Gemilang Pratama

2. Dzaki Devsi Pratama

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah.....	3
1.3 Manfaat Masalah.....	3
BAB II	4
LANDASAN TEORI	4
2.1 Pengolahan Citra Digital dan Ruang Warna RGB	4
2.2 Segmentasi Citra Berbasis Warna dan Thresholding	4
2.3 Analisis Distribusi Histogram Citra	5
2.4 Peningkatan Kualitas Citra (<i>Image Enhancement</i>) dan CLAHE.....	5
BAB III.....	8
HASIL	8
BAB IV.....	25
PENUTUP	25
DAFTAR PUSTAKA.....	26

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- a) Bagaimana merancang algoritma menggunakan pustaka OpenCV dan NumPy untuk mendeteksi, memisahkan, dan mengisolasi spektrum warna spesifik (merah, hijau, biru) pada sebuah citra teks?
- b) Bagaimana menganalisis perubahan distribusi nilai intensitas piksel antara citra asli dan citra hasil segmentasi warna menggunakan representasi grafik histogram?
- c) Bagaimana menentukan nilai ambang batas (*threshold margin*) yang paling optimal untuk menghindari *noise* saat memisahkan warna dominan dari latar belakang citra?
- d) Bagaimana mengatasi masalah degradasi visual akibat kondisi pencahayaan *backlight* (kekurangan paparan cahaya atau *underexposed*) pada foto menggunakan metode modifikasi kecerahan (*brightness*) dan ekualisasi kontras adaptif (CLAHE)?

1.2 Tujuan Masalah

- a) Mengimplementasikan algoritma pemisahan kanal ruang warna (RGB) dan *thresholding* biner untuk mengekstraksi teks berdasarkan target warna tertentu secara presisi.
- b) Menganalisis dan menginterpretasikan grafik histogram untuk memvalidasi keberhasilan proses *masking* dan perubahan intensitas cahaya pada citra digital.
- c) Menemukan dan mengevaluasi parameter nilai ambang batas (*threshold*) operasi logika matriks yang paling tepat untuk mendeteksi warna pigmen tinta tanpa terganggu oleh gradasi warna latar belakang.
- d) Menerapkan dan membandingkan efektivitas metode peningkatan kecerahan linier dan metode peningkatan kontras lokal (CLAHE) dalam memulihkan (*recovery*) detail visual serta tekstur pada foto malam hari yang mengalami kondisi *backlight*.

1.3 Manfaat Masalah

Manfaatnya memberikan pemahaman praktis yang mendalam bagi mahasiswa di bidang keilmuan Teknik Informatika mengenai operasi komputasi matriks piksel, manipulasi ruang warna, dan fundamental algoritma *Computer Vision* menggunakan bahasa pemrograman Python. Dan hal ini menjadi landasan pengetahuan teknis untuk rekayasa perangkat lunak yang lebih kompleks di masa depan. Algoritma deteksi warna dapat diimplementasikan pada sistem pemindai dokumen pintar (*Optical Character Recognition/OCR*), sementara algoritma perbaikan kontras (CLAHE) dapat diaplikasikan secara langsung pada pengembangan fitur perbaikan foto otomatis di berbagai platform media digital.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital dan Ruang Warna RGB

Pengolahan citra digital adalah serangkaian manipulasi matematis pada citra menggunakan algoritma komputasi. Tahapan ini melibatkan proses digitalisasi dari sinyal fisik analog menjadi kumpulan data diskrit melalui mekanisme *sampling* dan kuantisasi. Proses ini tidak hanya bertujuan untuk sekadar meningkatkan kualitas visual sebuah gambar, tetapi juga untuk melakukan ekstraksi informasi tingkat lanjut agar data visual dapat dianalisis secara otomatis oleh sistem. Dengan demikian, disiplin ilmu ini menjadi jembatan utama dan prasyarat mutlak bagi pengembangan teknologi kecerdasan buatan, seperti *computer vision* dan sistem pengenalan pola. Sebuah citra digital pada dasarnya direpresentasikan oleh komputer sebagai matriks angka dua dimensi atau tiga dimensi. Matriks ini pada hakikatnya adalah susunan array yang terdiri dari baris dan kolom yang merepresentasikan resolusi spasial dari gambar tersebut. Dalam komputasi modern, representasi spasial ini dipetakan dalam koordinat piksel, di mana setiap titik menyimpan nilai intensitas tertentu yang siap diproses secara komputasional menggunakan berbagai bahasa pemrograman dan operasi matriks. Pemrosesan tingkat tinggi ini sering kali difasilitasi oleh pustaka perangkat lunak komputasi array yang mampu menangani dan mengeksekusi jutaan kalkulasi titik secara paralel dalam sepersekian detik.

Standar ruang warna yang paling esensial dan umum digunakan adalah RGB (Red, Green, Blue). Sistem ini diadopsi secara luas karena meniru langsung cara kerja organ mata manusia, khususnya pada sel reseptor kerucut yang secara alami sensitif terhadap panjang gelombang cahaya merah, hijau, dan biru. Ruang warna ini bertindak sebagai model warna aditif utama yang mendasari cara kerja teknologi layar digital, monitor, dan sensor penangkap gambar modern. Disebut sebagai model warna aditif karena warna-warna baru diciptakan dengan menambahkan atau memancarkan gelombang cahaya ke dalam ruang yang asalnya gelap, sebuah prinsip yang sangat berlawanan dengan model warna subtraktif (seperti CMYK) yang menyerap cahaya dan digunakan pada industri percetakan tinta. Dalam model ini, setiap titik piksel dibentuk dari kombinasi tiga matriks intensitas cahaya dasar yang memiliki rentang nilai kuantisasi 8-bit, yaitu dari 0 hingga 255. Batas angka 8-bit ini bersumber dari perhitungan arsitektur sistem biner. Secara spesifik, nilai 0 pada sebuah saluran menunjukkan tidak adanya emisi cahaya (warna hitam pekat), sementara nilai maksimum 255 merepresentasikan intensitas cahaya yang paling terang. Sebagai contoh, jika matriks koordinat menunjuk pada nilai maksimal di ketiga salurannya sekaligus (255, 255, 255), maka layar akan memancarkan warna putih murni. Perpaduan matematis dari ketiga matriks intensitas ini memungkinkan sistem untuk merender hingga lebih dari 16,7 juta variasi kombinasi warna yang akurat. Jumlah masif ini diperoleh secara logis dari perkalian variasi tiap salurannya.

Pemahaman mengenai dekomposisi matriks RGB ini merupakan fundamental matematika yang digunakan untuk melakukan operasi aritmatika piksel, seperti isolasi dominansi warna pada suatu objek gambar. Dengan memisahkan citra ke dalam saluran warna tunggalnya (hanya saluran merah, hijau, atau biru saja), kita dapat mengaplikasikan berbagai kalkulasi tingkat lanjut. Misalnya, sebuah objek yang secara visual berwarna kuning dapat dideteksi dan diekstraksi oleh mesin dengan cara menganalisis irisan data di mana nilai saluran merah dan hijau saling bertumpang tindih dengan intensitas tinggi, sementara saluran birunya berada di angka yang sangat rendah. Hal ini membuka jalan bagi teknik-teknik pemrosesan lain yang lebih kompleks, seperti transformasi warna ke tingkat keabuan (grayscale), peningkatan kontras, deteksi tepi objek menggunakan konvolusi kernel, hingga manipulasi algoritma menggunakan logika boolean yang sangat dibutuhkan dalam proses segmentasi data visual. Operasi

transformasi *grayscale* bekerja dengan cara merepresentasikan ulang dimensi citra menjadi saluran tunggal melalui perhitungan rata-rata atau pembobotan proporsional dari ketiga saluran matriks RGB.

2.2 Segmentasi Citra Berbasis Warna dan Thresholding

Segmentasi citra adalah proses krusial untuk memisahkan objek target dari latar belakangnya. Tahapan ini merupakan landasan utama dan langkah awal dalam sistem *computer vision* serta analisis citra otomatis. Keberhasilan ekstraksi fitur dan pengenalan pola pada tahap komputasi selanjutnya sangat bergantung pada seberapa akurat pemisahan objek ini dilakukan. Dengan membuang informasi latar belakang yang tidak relevan atau yang dianggap sebagai *noise*, beban komputasi menjadi jauh lebih ringan karena proses analisis hanya difokuskan pada wilayah minat (*Region of Interest* / ROI) yang spesifik.

Dalam pendekatan ekstraksi warna, segmentasi dilakukan dengan mencari selisih batas toleransi antar kanal warna RGB. Teknik ini mengeksploitasi karakteristik spektral warna yang unik pada sebuah objek. Penentuan selisih batas toleransi ini biasanya membutuhkan analisis terhadap rentang nilai piksel tertentu, yang dirancang sedemikian rupa agar sistem komputer dapat mengenali warna target secara konsisten, meskipun terdapat sedikit variasi intensitas akibat perubahan pencahayaan pada citra asli.

Jika intensitas suatu piksel memenuhi syarat dominansi logika matematika (misalnya, nilai warna Biru jauh lebih besar daripada Merah dan Hijau), maka dilakukan operasi Thresholding Biner. Syarat dominansi ini diimplementasikan dengan mengevaluasi kondisi setiap titik matriks menggunakan operator *boolean* dan perbandingan kondisional. Evaluasi secara komputasional ini akan menyaring citra piksel demi piksel, memisahkan secara tegas mana bagian matriks yang benar-benar mewakili objek dan mana yang murni merupakan bagian dari lingkungan sekitarnya.

Setelah kondisi logika tersebut dievaluasi, sistem akan menghasilkan keluaran akhir berupa peta biner yang sangat terstruktur. Thresholding akan mengonversi piksel yang memenuhi syarat batas ambang menjadi nilai piksel maksimum (255 atau putih absolut) yang berfungsi sebagai masking, dan mengubah piksel yang tidak memenuhi syarat menjadi nilai minimum (0 atau hitam absolut). Citra biner hitam-putih hasil dari proses *masking* ini kemudian dapat digunakan sebagai cetakan. Jika citra *masking* ini dikalikan kembali menggunakan operasi *bitwise AND* dengan matriks citra RGB aslinya, maka hasil akhirnya hanya akan menampilkan objek warna yang diisolasi dengan latar belakang yang telah bersih sepenuhnya.

2.3 Analisis Distribusi Histogram Citra

Histogram citra merupakan representasi grafik yang menjabarkan secara visual distribusi frekuensi nilai intensitas piksel dalam sebuah gambar digital. Dalam disiplin pengolahan citra digital, alat statistik ini sangat fundamental karena memberikan ringkasan informasi global mengenai komposisi matriks penyusun gambar tanpa harus menganalisis struktur spasial atau letak pikselnya secara langsung. Pemetaan probabilitas kemunculan nilai warna ini menjadi acuan penting dalam proses prapemrosesan.

Pada grafik histogram, sumbu horizontal (X) mewakili tingkat kecerahan warna (0 untuk hitam pekat hingga 255 untuk putih murni), sementara sumbu vertikal (Y) merepresentasikan jumlah kumulatif piksel yang berada pada tingkat kecerahan tersebut. Rentang nilai 0 hingga 255 ini berasal dari kedalaman warna standar 8-bit yang umum diimplementasikan pada komputasi visual. Untuk citra berskala keabuan, histogram akan menampilkan satu kurva intensitas cahaya. Namun, untuk citra berwarna, histogram dapat dipecah menjadi tiga grafik terpisah untuk mengevaluasi masing-masing saluran warna merah, hijau, dan biru secara independen.

Melalui histogram, karakteristik pencahayaan citra dapat dievaluasi secara matematis. Evaluasi numerik ini memungkinkan kita untuk mengukur berbagai parameter visual secara objektif, seperti rentang dinamis (*dynamic range*), tingkat kecerahan keseluruhan (*overall brightness*), dan rasio kontras. Analisis statistik dari sebaran frekuensi ini sering kali dijadikan dasar pengambilan keputusan sebelum mengaplikasikan operasi perbaikan citra seperti *Histogram Equalization*, yang bertujuan untuk meratakan distribusi piksel demi mendapatkan kualitas gambar yang lebih optimal.

2.4 Peningkatan Kualitas Citra (*Image Enhancement*) dan CLAHE

Image Enhancement bertujuan untuk merekonstruksi dan memperjelas fitur spesifik dari suatu citra tanpa menambah informasi baru di dalamnya. Proses prapemrosesan ini berfokus pada perbaikan kualitas visual data agar lebih mudah diinterpretasikan oleh mata manusia maupun diekstraksi oleh algoritma *computer vision*. Prinsip utamanya adalah mengoptimalkan nilai piksel yang sudah ada sehingga detail yang tersembunyi dapat direpresentasikan dengan lebih baik, tanpa memanipulasi keaslian data. Salah satu permasalahan umum adalah fenomena *backlight*, di mana objek utama menjadi sangat gelap akibat dominasi cahaya latar belakang. Kondisi pencahayaan dengan rentang dinamis yang ekstrem ini terjadi karena sensor kamera terpaksa menyesuaikan eksposur dengan cahaya latar yang sangat kuat, sehingga informasi tekstur dan bentuk pada objek utama tidak terekam sempurna dan jatuh ke area bayangan (*shadow*).

Meskipun modifikasi peningkatan kecerahan linier dapat menerangkan objek, metode ini sering kali memicu kerusakan visual berupa *over-exposure* pada area yang sudah terang. Peningkatan intensitas secara linier akan memaksakan pergeseran seluruh grafik histogram ke sisi kanan secara serentak. Akibatnya, piksel-piksel pada area latar belakang yang pada dasarnya sudah memiliki intensitas tinggi akan menabrak batas maksimal kuantisasi (nilai 255), kehilangan detail aslinya, dan berubah menjadi putih datar yang merusak kualitas informasi citra.

Untuk mengatasi limitasi tersebut, metode Contrast Limited Adaptive Histogram Equalization (CLAHE) digunakan. Teknik ini menawarkan pendekatan matematis tingkat lanjut yang mampu menyeimbangkan rentang dinamis citra dengan memperhitungkan kondisi intensitas di sekitar masing-masing piksel.

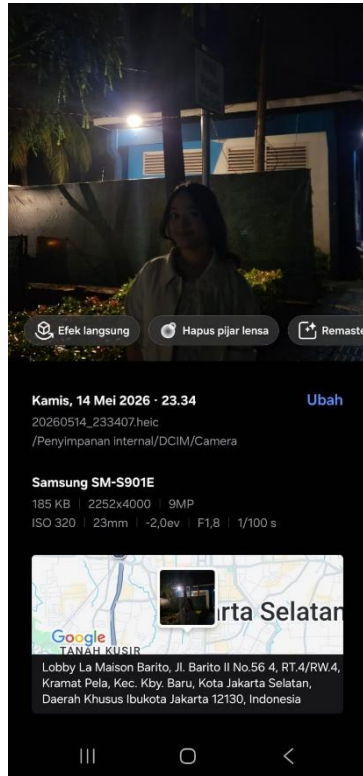
Berbeda dengan pemerataan histogram tradisional yang bersifat global, algoritma CLAHE bekerja secara lokal. Pada metode *Global Histogram Equalization* (GHE), fungsi transformasi dihitung berdasarkan satu distribusi kumulatif keseluruhan gambar, yang berisiko membuat area terang menjadi terlalu menyilaukan dan area gelap menjadi kehilangan kedalaman. Sebagai solusinya, CLAHE membagi gambar ke dalam beberapa blok area kecil dan melakukan peningkatan kontras secara spesifik pada masing-masing area tersebut. Blok-blok pemrosesan ini, yang sering direpresentasikan dalam bentuk *grid* atau *tiles* (umumnya berukuran 8x8 matriks piksel), dianalisis histogramnya secara independen. Setelah setiap *tile* diperbaiki kontrasnya, algoritma ini kemudian menggunakan teknik interpolasi bilinear untuk menjahit dan menyatukan kembali batas antar blok tersebut agar tidak meninggalkan garis artefak yang patah-patah pada gambar akhir.

Meskipun pemrosesan berbasis blok lokal ini sangat efektif, pendekatan ini rentan terhadap kelemahan saat menjumpai blok yang memiliki intensitas warna yang seragam, di mana histogramnya akan membentuk puncak lonjakan frekuensi yang sangat sempit dan tinggi. Jika disamakan begitu saja, hal ini akan mengekspos dan memperkuat *noise* mikroskopis menjadi sangat jelas. Agar tidak terjadi penguatan gangguan pada area piksel yang homogen, CLAHE menggunakan parameter Clip Limit untuk memotong batas puncak histogram lokal dan mendistribusikan ulang sisa nilai pikselnya, sehingga

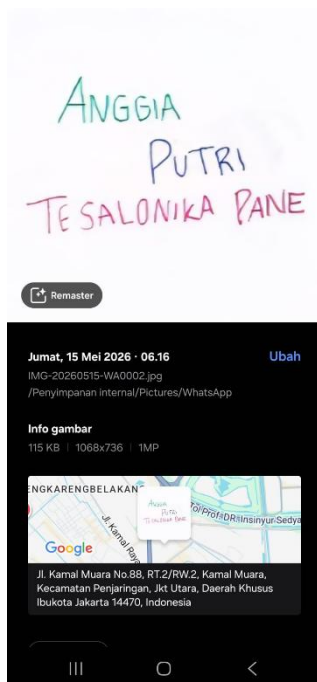
menghasilkan citra dengan perbaikan visibilitas bayangan yang tajam dan natural. Potongan nilai piksel yang melewati batas *Clip Limit* ini tidak dibuang, melainkan dibagi rata ke seluruh elemen nilai tingkat keabuan lainnya. Mekanisme cerdas ini memastikan kurva pemetaan tidak terlalu curam, sehingga detail pada objek yang terkena *backlight* dapat diangkat secara maksimal tanpa memunculkan bintik-bintik *noise* yang mengorbankan kualitas estetika maupun akurasi data.

BAB III HASIL

Gambar tangkapan layar awal:



Gambar awal yang akan di deteksi:



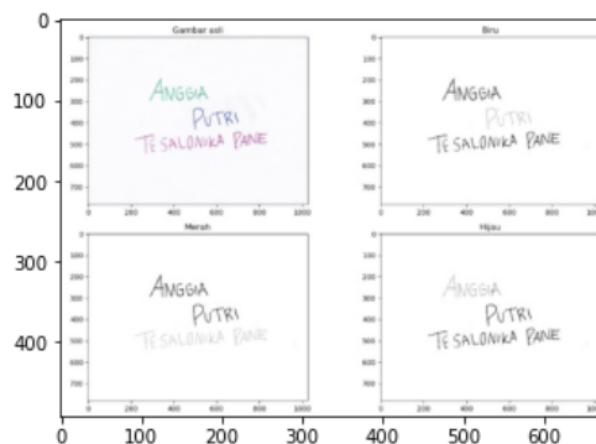
Soal nomor 1

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

```
In [3]: img = cv2.imread('tulisan.jpeg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
In [4]: plt.imshow(img_rgb)
```

```
Out[4]: <matplotlib.image.AxesImage at 0x209897af160>
```



Blok kode In [1] dan In [3], program melakukan pemuatan pustaka esensial (terutama OpenCV dan Matplotlib), membaca *file* gambar bernama 'tulisan.jpeg', serta mengonversi format ruang warnanya dari BGR (format bawaan arsitektur OpenCV) menjadi RGB. Konversi ini wajib dilakukan agar warna direpresentasikan secara akurat dan tidak terdistorsi ketika diproyeksikan oleh Matplotlib. Perintah `plt.imshow(img_rgb)` memanggil fungsi dari modul Matplotlib untuk merender dan memproyeksikan matriks data piksel yang tersimpan dalam variabel `img_rgb` menjadi grafik visual ke layar antarmuka. Teks `<matplotlib.image.AxesImage at ...>` yang muncul tepat di bawah baris kode merupakan pesan log standar dari lingkungan kerja (seperti Jupyter Notebook). Pesan tersebut mengonfirmasi bahwa sebuah objek citra telah berhasil dialokasikan dan diregistrasi di dalam ruang memori sistem pada alamat memori tertentu sebelum dirender ke layar visual. Hasil visual yang dijalankan oleh fungsi `imshow` pada tahap ini bukanlah citra tulisan mentah, melainkan sebuah gambar yang di dalamnya sudah memuat kompilasi tata letak empat panel (Gambar asli, Biru, Merah, dan Hijau). Hal ini memberikan kesimpulan teknis bahwa *file* 'tulisan.jpeg' yang diimpor ke dalam variabel `img` sejatinya adalah gambar hasil tangkapan layar, bukan citra sumber murni.

```
In [8]: print(img.shape)
(418, 547, 3)
```

Blok kode In [8] menjalankan perintah `print(img.shape)` yang berfungsi untuk menginspeksi dimensi atau ukuran matriks dari citra digital yang telah dimuat ke dalam memori variabel `img`. Dalam pengolahan citra menggunakan pustaka NumPy, setiap gambar direpresentasikan sebagai matriks *array multidimensi*, dan atribut `.shape` digunakan untuk mengekstraksi informasi struktural dari matriks tersebut. Hasil dari perintah ini berupa *tuple* (418, 547, 3) yang secara presisi mendeskripsikan struktur tiga dimensi dari citra yang sedang diproses. Angka pertama, yaitu **418**, mengindikasikan tinggi citra (*height*), yang berarti matriks

citra tersebut memiliki 418 baris piksel vertikal. Angka kedua, yaitu **547**, menunjukkan lebar citra (*width*), yang berarti terdapat 547 kolom piksel horizontal. Total resolusi gambar tersebut adalah 418 x 547 piksel. Sementara itu, angka ketiga, yaitu **3**, merepresentasikan kedalaman atau jumlah kanal warna. Nilai 3 ini mengonfirmasi bahwa data yang sedang diolah merupakan citra berwarna (bukan *grayscale*).

```
In [9]: r_diff = cv2.subtract(img_rgb[:, :, 0], cv2.max(img_rgb[:, :, 1], img_rgb[:, :, 2]))
        g_diff = cv2.subtract(img_rgb[:, :, 1], cv2.max(img_rgb[:, :, 0], img_rgb[:, :, 2]))
        b_diff = cv2.subtract(img_rgb[:, :, 2], cv2.max(img_rgb[:, :, 0], img_rgb[:, :, 1]))

In [10]: _, r_mask = cv2.threshold(r_diff, 20, 255, cv2.THRESH_BINARY)
         _, g_mask = cv2.threshold(g_diff, 20, 255, cv2.THRESH_BINARY)
         _, b_mask = cv2.threshold(b_diff, 20, 255, cv2.THRESH_BINARY)

In [11]: def proc_res(t_mask, ori_img):
         out = np.full(ori_img.shape[:2], 255, dtype=np.uint8)

         gray = cv2.cvtColor(ori_img, cv2.COLOR_RGB2GRAY)
         _, txt_mask = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY_INV)

         out[txt_mask > 0] = 0
         out[t_mask > 0] = 200

         return out

r_res = proc_res(r_mask, img_rgb)
g_res = proc_res(g_mask, img_rgb)
b_res = proc_res(b_mask, img_rgb)

# Display
ttl = ['Gambar asli', 'Biru', 'Merah', 'Hijau']
imgs = [img_rgb, b_res, r_res, g_res]

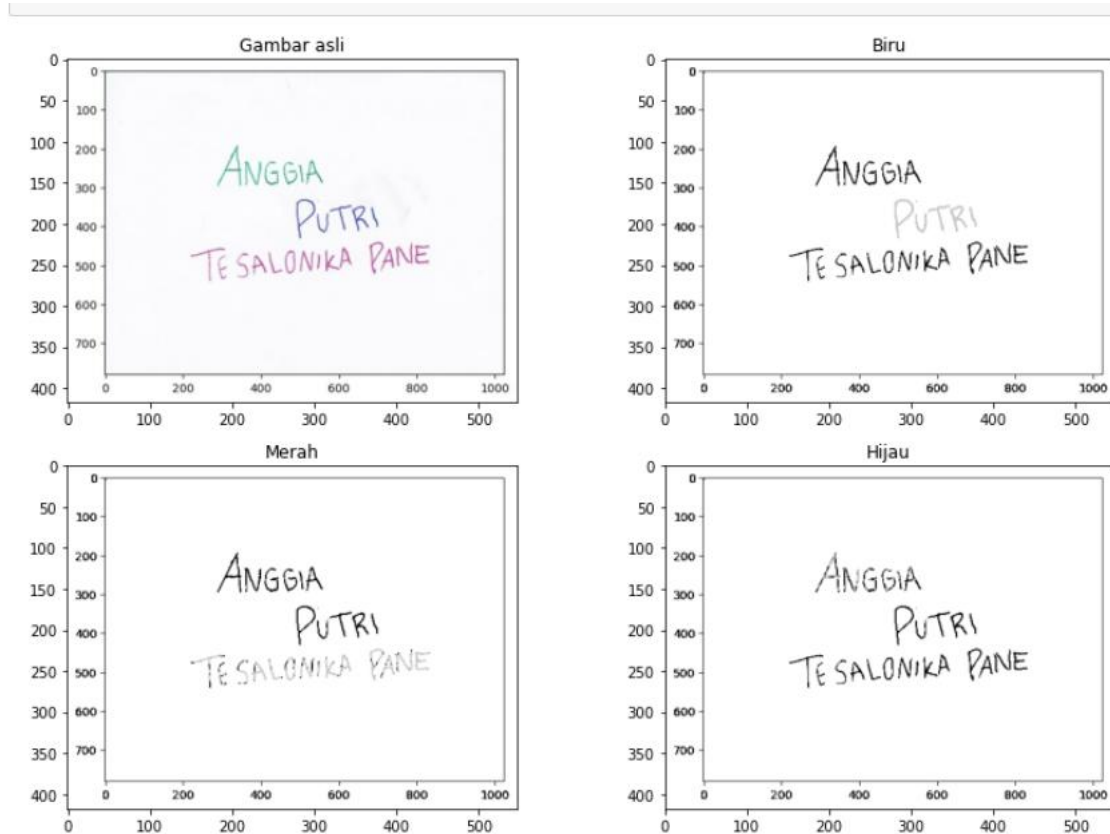
plt.figure(figsize=(12,8))

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(imgs[i], cmap='gray' if i > 0 else None)
    plt.title(ttl[i])
    plt.axis('on')

plt.tight_layout()
plt.show()
```

Tahap pertama bertujuan untuk mengisolasi spektrum warna merah, hijau, dan biru murni dari citra asli. Program menggunakan operasi aritmatika piksel matriks dengan fungsi `cv2.subtract` dan `cv2.max`. Logika komputasinya adalah dengan mengurangi intensitas satu kanal warna target dengan nilai intensitas maksimum dari dua kanal warna lainnya pada titik piksel yang sama. Sebagai contoh, pada pembentukan variabel `r_diff` (selisih merah), program mengambil nilai dari kanal merah (indeks 0 pada format RGB), lalu mengurangnya dengan nilai tertinggi antara kanal hijau (indeks 1) dan biru (indeks 2). Melalui algoritma ini, piksel yang berwarna putih atau abu-abu (di mana nilai R, G, dan B hampir seimbang) akan menghasilkan nilai selisih mendekati nol. Sebaliknya, piksel yang dominan warna merah akan menghasilkan nilai selisih yang tinggi. Hal yang sama diterapkan untuk variabel `g_diff` dan `b_diff`. Setelah nilai dominasi warna didapatkan, program melakukan proses segmentasi menggunakan metode pengambungan (*thresholding*) biner melalui fungsi `cv2.threshold`. Nilai ambang batas (*threshold value*) ditetapkan sebesar 20. Jika nilai selisih warna pada suatu piksel (dari tahap sebelumnya) lebih besar dari 20, maka piksel tersebut akan dikonversi menjadi putih absolut (nilai 255). Sebaliknya, jika bernilai 20 ke bawah, akan diubah menjadi hitam (nilai 0). Proses ini menghasilkan tiga buah *masking* biner (`r_mask`, `g_mask`, dan `b_mask`) yang berfungsi sebagai peta penunjuk letak koordinat tulisan berwarna merah, hijau, dan biru secara spesifik pada gambar asli. Fungsi `proc_res` adalah inti dari manipulasi representasi warna untuk menjawab kebutuhan analisis. Fungsi ini menerima dua parameter: *masking* warna target (`t_mask`) dan citra asli (`ori_img`).

Setelah fungsi `proc_res` dieksekusi untuk ketiga warna, langkah terakhir adalah memproyeksikan seluruh gambar menggunakan antarmuka pustaka Matplotlib. Program menginisiasi figur berukuran 12x8 inci (`plt.figure`) dan menggunakan struktur perulangan *for* untuk menampilkan empat citra secara terstruktur dalam grid tata letak 2x2. Parameter penentu pada tahapan ini adalah pengondisian `cmap='gray'` if $i > 0$ else `None`. Parameter ini menginstruksikan sistem agar indeks gambar ke-0 (citra asli) ditampilkan dalam format warna standar RGB, sementara gambar hasil ekstraksi (indeks ke-1 hingga ke-3) secara paksa dirender ke



dalam peta warna *grayscale*

Gambar tersebut merupakan visualisasi akhir dari tahapan pemrosesan citra digital yang disajikan dalam bentuk *grid* matriks 2x2. Visualisasi ini bertujuan untuk membandingkan citra sumber (*input*) dengan tiga citra hasil segmentasi dan ekstraksi berdasarkan dominasi ruang warna (Biru, Merah, dan Hijau). Berikut adalah rincian analisis dari setiap panel keluaran tersebut:

1. Gambar Asli (Kuadran Kiri Atas)

Panel ini menampilkan representasi citra mentah sebelum diberikan perlakuan algoritma. Pada citra ini, terdapat teks tulisan tangan dengan tiga komposisi warna tinta yang berbeda, yaitu kata "ANGGIA" yang ditulis menggunakan warna hijau toska, kata "PUTRI" menggunakan warna biru, dan kalimat "TESALONIKA PANE" yang ditulis menggunakan warna merah muda. Gambar ini menjadi parameter dasar (*baseline*) untuk mengevaluasi akurasi deteksi warna pada tahapan selanjutnya.

2. Visualisasi Deteksi Biru (Kuadran Kanan Atas)

Pada panel "Biru", program menampilkan hasil penyeleksian khusus untuk spektrum warna biru. Sesuai dengan algoritma modifikasi intensitas yang telah dirancang sebelumnya, piksel yang terdeteksi memiliki dominasi warna biru (yaitu pada kata "PUTRI") direpresentasikan dengan warna abu-abu terang (nilai piksel 200), sehingga secara visual tampak memudar dan hampir menyatu dengan latar belakang kertas yang berwarna putih murni. Sebaliknya, teks yang tidak mengandung

warna biru dominan (yaitu "ANGGIA" dan "TESALONIKA PANE") secara paksa dikonversi menjadi warna hitam pekat (nilai piksel 0).

3. Visualisasi Deteksi Merah (Kuadran Kiri Bawah)

Pada panel "Merah", sistem memfokuskan deteksi pada spektrum warna merah. Hasilnya menunjukkan bahwa kalimat "TESALONIKA PANE" (yang pada citra asli berwarna merah muda) berhasil dideteksi sebagai target warna merah. Teks tersebut kemudian diubah intensitasnya menjadi abu-abu terang, sementara kata "ANGGIA" (hijau) dan "PUTRI" (biru) diabaikan oleh *masking* merah dan diubah menjadi warna hitam pekat.

4. Visualisasi Deteksi Hijau (Kuadran Kanan Bawah)

Pada panel terakhir ini, algoritma melakukan isolasi terhadap warna hijau. Terlihat dengan jelas bahwa kata "ANGGIA" merespons pendeteksian ini dan tervisualisasi dengan warna abu-abu terang yang memudar. Di saat yang bersamaan, sisa teks lainnya yang berwarna biru dan merah muda diubah menjadi warna hitam pekat karena tidak memenuhi nilai ambang batas dominasi warna hijau.

```
In [12]: r_res = r_res.astype(np.uint8)
         g_res = g_res.astype(np.uint8)
         b_res = b_res.astype(np.uint8)

         fig, axs = plt.subplots(4, 2, figsize=(14, 16))

         axs[0, 0].imshow(img_rgb)
         axs[0, 0].set_title('Citra Kontras')

         axs[0, 1].hist(img_rgb.ravel(), 256, range=[0, 256])
         axs[0, 1].set_title('Histogram Citra Kontras')

         axs[1, 0].imshow(b_res, cmap='gray')
         axs[1, 0].set_title('Biru')

         axs[1, 1].hist(b_res.ravel(), 256, range=[0, 256])
         axs[1, 1].set_title('Histogram Biru')

         axs[2, 0].imshow(r_res, cmap='gray')
         axs[2, 0].set_title('Merah')

         axs[2, 1].hist(r_res.ravel(), 256, range=[0, 256])
         axs[2, 1].set_title('Histogram Merah')
         |
         axs[3, 0].imshow(g_res, cmap='gray')
         axs[3, 0].set_title('Hijau')

         axs[3, 1].hist(g_res.ravel(), 256, range=[0, 256])
         axs[3, 1].set_title('Histogram Hijau')

         plt.tight_layout()
         plt.show()
```

Blok kode In [12] berfungsi untuk merancang tata letak visualisasi akhir yang komprehensif, yaitu menyandingkan setiap gambar (baik citra asli maupun hasil ekstraksi) dengan grafik histogramnya masing-masing.

Tahap pertama dimulai dengan standardisasi tipe data matriks. Fungsi `.astype(np.uint8)` digunakan untuk mengonversi secara eksplisit variabel hasil pemrosesan (`r_res`, `g_res`, dan `b_res`) ke dalam tipe data *Unsigned Integer 8-bit*. Langkah ini sangat krusial dalam pengolahan citra digital karena pustaka visualisasi dan analisis matriks berasumsi bahwa nilai intensitas piksel citra standar selalu berada dalam rentang bilangan bulat 0 hingga 255. Jika tidak dikonversi, komputasi pembuatan histogram dapat mengalami *error* atau interpretasi nilai rentang warna yang salah.

Selanjutnya, program menginisialisasi kerangka visual (*figure*) menggunakan perintah `plt.subplots(4, 2, figsize=(14, 16))`. Perintah ini menginstruksikan pustaka Matplotlib untuk

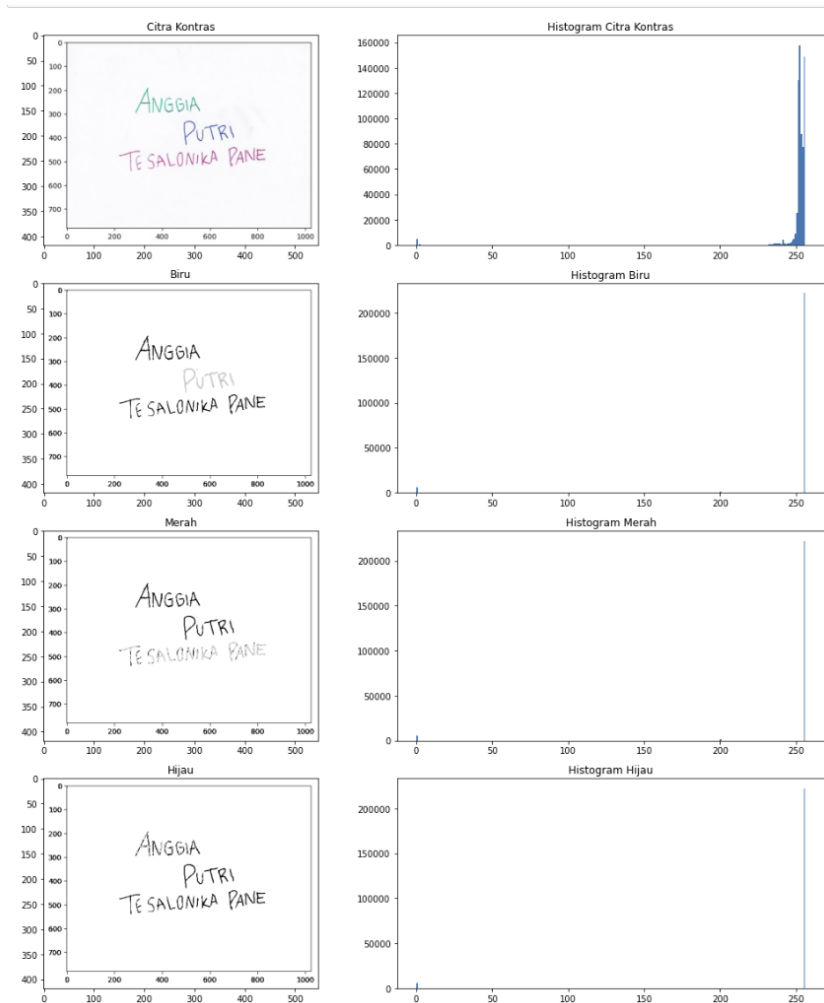
membuat sebuah kanvas besar berukuran 14x16 inci yang terbagi ke dalam *grid* matriks berukuran 4 baris dan 2 kolom. Empat baris tersebut mewakili masing-masing kategori citra, sementara dua kolom memisahkan fungsi tampilan: kolom pertama (indeks 0) untuk menampilkan gambar, dan kolom kedua (indeks 1) untuk menampilkan grafik histogram. Variabel `axs` berfungsi sebagai *array* koordinat untuk menempatkan setiap plot secara presisi pada *grid* tersebut.

Bagian inti dari kode ini adalah operasi pengulangan pola penempatan visual pada setiap baris matriks `axs`. Pada kolom gambar (`axs[... , 0]`), program menggunakan fungsi `.imshow()` untuk memproyeksikan matriks citra, dengan penambahan parameter `cmap='gray'` khusus untuk citra hasil ekstraksi agar ditampilkan dalam format derajat keabuan.

Pada kolom grafik (`axs[... , 1]`), program mengeksekusi perhitungan distribusi piksel menggunakan fungsi `.hist()`. Terdapat tiga komponen utama dalam fungsi komputasi histogram ini:

- a) `.ravel()`: Karena gambar aslinya berbentuk matriks 2D atau 3D, fungsi ini digunakan untuk "meratakan" (*flatten*) seluruh nilai matriks ke dalam format *array* 1D. Hal ini diwajibkan karena fungsi histogram hanya dapat menghitung frekuensi kemunculan nilai dari satu deret angka linear.
- b) 256: Merupakan jumlah *bins* atau keranjang data. Artinya, grafik sumbu X akan dibagi menjadi 256 titik intensitas warna.
- c) `range=[0, 256]`: Membatasi perhitungan frekuensi piksel secara absolut hanya pada rentang nilai kecerahan warna yang valid, yaitu dari nilai 0 (hitam pekat) hingga batas 255 (putih murni). Parameter ini mencegah masuknya nilai anomali (*noise* di luar batas) ke dalam grafik.

Tahap akhir ditutup dengan pemanggilan `.tight_layout()` yang berfungsi untuk melakukan penyesuaian tata letak otomatis agar elemen-elemen pada figur (seperti teks judul sumbu, angka margin, dan gambar) tidak saling bertumpuk (*overlapping*). Setelah tata letak dirapikan, perintah `plt.show()` dieksekusi untuk merender keseluruhan kanvas visual tersebut ke layar.



Gambar keluaran tersebut menyajikan perbandingan antara citra spasial dengan representasi grafik histogramnya, yang menunjukkan distribusi frekuensi nilai intensitas piksel dari rentang 0 (hitam pekat) hingga 255 (putih murni). Pada histogram citra kontras atau citra asli, terlihat grafik mendominasi secara fluktuatif di area intensitas tinggi (mendekati 255) yang membentuk puncak melebar. Hal ini merepresentasikan warna latar belakang kertas yang mendominasi gambar dengan kondisi pencahayaan dunia nyata yang tidak merata, sehingga menghasilkan gradasi warna putih hingga abu-abu terang. Sementara itu, sebaran frekuensi yang sangat kecil dan tipis di area intensitas menengah hingga rendah merepresentasikan piksel-piksel dari tulisan tangan berwarna hijau, biru, dan merah muda yang memiliki tingkat intensitas kegelapan bervariasi.

Karakteristik histogram mengalami perubahan yang sangat drastis pada ketiga citra hasil ekstraksi ("Biru", "Merah", dan "Hijau"). Grafik yang sebelumnya menyebar secara kontinu berubah menjadi sangat diskrit dan hanya terpusat pada tiga titik intensitas utama, yang secara langsung membuktikan bahwa algoritma manipulasi piksel telah berhasil diaplikasikan. Pertama, terdapat sebuah garis vertikal yang menjulang sangat tinggi tepat di titik nilai 255, yang memvalidasi bahwa latar belakang citra bergradasi telah diganti sepenuhnya menjadi kanvas putih murni absolut. Kedua, terdapat kemunculan frekuensi piksel di titik 0 (hitam pekat) yang merepresentasikan tulisan di luar target warna yang dipaksa menjadi gelap. Ketiga, terdapat sebaran piksel bernilai 200 (abu-abu terang) yang merepresentasikan kata atau tulisan yang berhasil dideteksi sesuai dengan warna targetnya masing-masing.

Secara keseluruhan, transformasi bentuk histogram dari yang bersifat kontinu pada citra asli menjadi diskrit pada citra keluaran memberikan pembuktian matematis yang kuat terhadap keberhasilan program. Pemusatan nilai piksel yang hanya berada di angka 0, 200, dan 255 memvalidasi bahwa tahapan

thresholding, pembentukan *masking* biner, dan klasifikasi teks warna telah dieksekusi dengan presisi tinggi dan berjalan persis seperti logika algoritma yang dirancang pada awal kode program.

```
In [13]: r_ch = img_rgb[:, :, 0].astype(np.int16)
g_ch = img_rgb[:, :, 1].astype(np.int16)
b_ch = img_rgb[:, :, 2].astype(np.int16)

n_res = np.zeros(img_rgb.shape[:2], dtype=np.uint8)

b_mask = np.where(
    (b_ch > r_ch + 30) &
    (b_ch > g_ch + 30),
    255, 0
).astype(np.uint8)

r_mask = np.where(
    (r_ch > g_ch + 30) &
    (r_ch > b_ch + 30),
    255, 0
).astype(np.uint8)

rb_res = cv2.bitwise_or(r_mask, b_mask)

g_mask = np.where(
    (g_ch > r_ch + 20) &
    (g_ch > b_ch + 20),
    255, 0
).astype(np.uint8)

rgb_res = cv2.bitwise_or(rb_res, g_mask)

ttl = ['NONE', 'BLUE', 'RED-BLUE', 'RED-GREEN-BLUE']
imgs = [n_res, b_mask, rb_res, rgb_res]

plt.figure(figsize=(12, 8))

for i in range(4):
    plt.subplot(2, 2, i + 1)

    plt.imshow(imgs[i], cmap='gray')

    plt.title(ttl[i], fontsize=12)
    plt.axis('on')

plt.tight_layout()

plt.show()
```

Logika Pemisahan Warna pada Kode Program

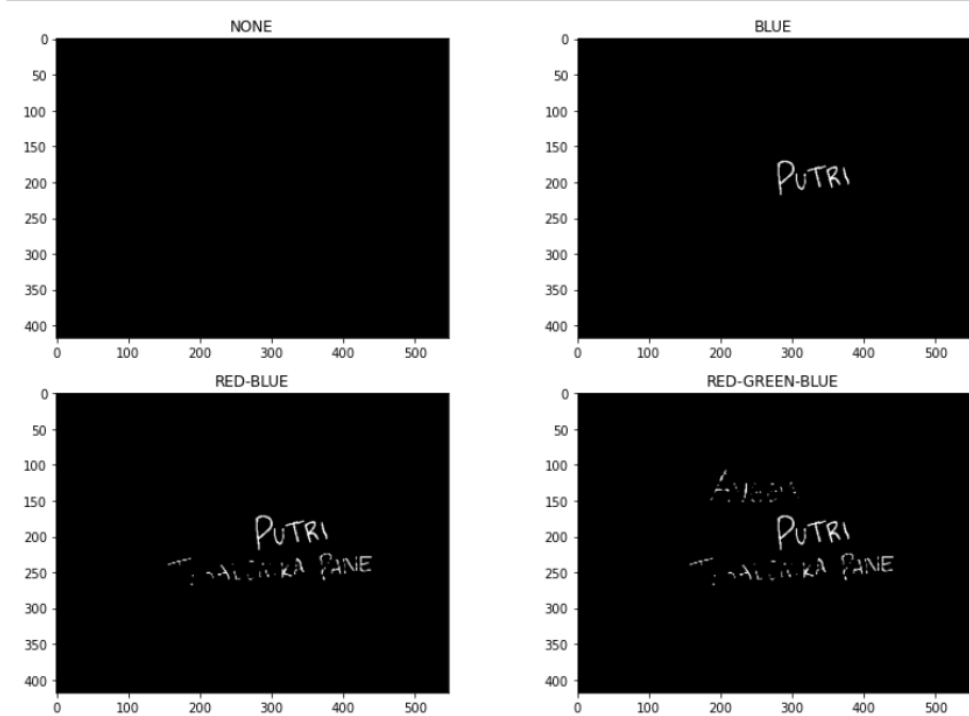
Berbeda dengan tahapan sebelumnya yang menggunakan fungsi bawaan `cv2.threshold`, pada blok kode ini, segmentasi warna dilakukan secara manual menggunakan operasi logika bersyarat (boolean) berbasis matriks dengan pustaka NumPy (`np.where`).

Langkah pertama yang dilakukan program adalah memisahkan citra ke dalam tiga kanal warna murni (Red, Green, Blue) dan mengonversi tipe datanya menjadi `np.int16` (*16-bit integer*). Konversi tipe data ini sangat krusial; karena program akan melakukan operasi penambahan nilai pada intensitas piksel, penggunaan *8-bit integer* bawaan berisiko mengalami nilai melebihi batas maksimal 255 dan kembali ke 0 yang akan mengacaukan perhitungan komputasi.

Nilai Ambang Batas (Threshold) yang di dapat Berdasarkan parameter penyeleksian di dalam blok kode, nilai ambang batas (*threshold margin*) yang digunakan untuk memisahkan setiap kategori warna adalah sebagai berikut:

- a) Warna Biru (`b_mask`)
Nilai ambang batas = 30
- b) Warna Merah (`r_mask`)
Nilai ambang batas = 30
- c) Warna Hijau (`g_mask`)
Nilai ambang batas = 20

Alasan Penentuan Nilai Ambang Batas, yaitu nilai ambang batas pada algoritma ini tidak berfungsi sebagai titik potong statis pada citra *grayscale*.



Hasil *output* memvisualisasikan proses penumpukan *masking* dari setiap warna secara bertahap menggunakan operasi `cv2.bitwise_or`. Operasi ini berfungsi seperti menumpuk kertas transparan, di mana jika ada piksel putih pada salah satu *masking*, maka hasil akhirnya akan tetap putih.

- Panel NONE
Menampilkan kanvas kosong (`n_res`) yang dideklarasikan dengan matriks bernilai 0 (hitam pekat) sebagai titik awal dasar.
- Panel BLUE
Menampilkan hasil *masking* biru secara tunggal. Hanya kata "PUTRI" yang terdeteksi dengan sangat baik dan bersih karena tinta biru sangat dominan.
- Panel RED-BLUE
Menumpuk *masking* biru dengan *masking* merah. Terlihat kalimat "TESALONIKA PANE" berhasil dimunculkan. Namun, karena kondisi pencahayaan dan tinta yang tidak solid, teks merah terlihat sedikit terputus-putus.
- Panel RED-GREEN-BLUE
Merupakan penggabungan akhir dengan memasukkan *masking* hijau. Kata "ANGGIA" berhasil divisualisasikan. Sesuai dengan konsekuensi dari penurunan ambang batas hijau menjadi 20 (yang dijelaskan pada poin 3), tulisan hijau terlihat sangat terdegradasi dan rentan terhadap *noise*, menandakan bahwa pigmen warna hijau pada citra asli merupakan yang paling sulit diisolasi di antara ketiga warna tersebut.

Soal nomor 2

Berikut merupakan foto asli dari gambar yang akan di olah.



```
In [15]: img_p = cv2.imread('foto.jpeg')
In [16]: image_photo_rgb = cv2.cvtColor(img_p, cv2.COLOR_BGR2RGB)
In [17]: plt.imshow(image_photo_rgb)
Out[17]: <matplotlib.image.AxesImage at 0x2098bf25d60>
```



Blok kode ini mendemonstrasikan tahapan paling awal dalam alur kerja pengolahan citra digital, yaitu proses akuisisi, prapemrosesan ruang warna, dan penayangan citra mentah. Pada tahap pertama, program menggunakan fungsi `cv2.imread('foto.jpeg')` untuk membaca *file* gambar dari media penyimpanan dan memuatnya ke dalam memori komputer sebagai matriks piksel pada variabel `img_p`. Karena arsitektur pembacaan bawaan dari pustaka OpenCV menyimpan saluran citra dalam format BGR (Blue, Green, Red), program wajib melakukan penyesuaian ruang warna menggunakan fungsi `cv2.cvtColor(img_p, cv2.COLOR_BGR2RGB)`. Proses konversi matematis ini memastikan bahwa urutan matriks warna diubah menjadi format standar RGB (Red, Green, Blue) yang disimpan dalam variabel `image_photo_rgb`, sehingga tidak terjadi distorsi atau warna yang tertukar ketika gambar diproyeksikan oleh pustaka visualisasi Matplotlib pada baris kode selanjutnya (`plt.imshow`).

Berdasarkan *output* visual yang dihasilkan, program telah berhasil merender citra secara akurat. Secara analitis, citra masukan tersebut merupakan sebuah foto yang diambil pada kondisi pencahayaan yang sangat minim (malam hari) dan mengalami masalah kekurangan pencahayaan. Subjek utama di area latar depan menjadi sangat gelap menyerupai siluet karena tidak mendapatkan iluminasi yang cukup, sementara terdapat sumber cahaya lampu yang terlampau terang di area latar belakang. Dari sudut pandang pengolahan citra, foto dengan tingkat kontras yang buruk dan distribusi piksel yang terpusat pada nilai intensitas sangat rendah (gelap) seperti ini mengindikasikan perlunya tahapan perbaikan kualitas citra. Pada tahapan selanjutnya, citra ini merupakan kandidat yang ideal untuk diproses menggunakan teknik modifikasi kecerahan, perbaikan kontras, atau pemerataan histogram agar detail visual pada subjek yang gelap dapat diungkapkan kembali.

```
In [18]: image_photo_gray = cv2.cvtColor(img_p, cv2.COLOR_BGR2GRAY)
```

Di konversi ke *grayscale*

```
In [19]: b_val = 50

img_bri = np.clip(
    image_photo_gray.astype(np.int32) + b_val,
    0,
    255
).astype(np.uint8)
```

```
In [20]: g_val = 2.5

lut = np.array([
    ((i / 255.0) ** (1.0 / g_val)) * 255
    for i in range(256)
]).astype(np.uint8)
```

Blok kode tersebut mengimplementasikan dua metode operasi titik (*point operation*) yang berbeda untuk melakukan perbaikan kualitas citra (*image enhancement*), khususnya untuk mengatasi masalah kekurangan pencahayaan (*underexposed*) pada foto malam hari sebelumnya. Pada sel kode pertama, program melakukan operasi peningkatan kecerahan linier dengan mendefinisikan nilai penambahan konstan sebesar 50 pada variabel `b_val`. Secara matematis, nilai ini ditambahkan ke seluruh piksel citra secara merata. Tahapan krusial pada kode ini terletak pada penggunaan `astype(np.int32)` sebelum proses penambahan, yang berfungsi untuk mencegah terjadinya galat komputasi *overflow* (kondisi di mana nilai piksel melampaui batas maksimal 255 dan kembali ke 0). Setelah penambahan selesai, fungsi `np.clip` digunakan untuk memaksa nilai piksel yang melebihi batas agar tetap tertahan di angka maksimal 255, sebelum akhirnya dikembalikan ke format matriks citra standar `np.uint8`. Hasil keluaran dari metode ini adalah citra yang secara keseluruhan menjadi lebih terang; subjek di area gelap akan lebih terlihat, namun metode ini memiliki kelemahan yaitu area yang sudah terang berisiko mengalami kehilangan detail warna karena nilainya terpotong di angka 255.

Pada sel kode kedua, program menyiapkan metode alternatif yang lebih canggih, yaitu Koreksi Gamma (*Gamma Correction*) non-linier dengan menetapkan nilai variabel `g_val` sebesar 2.5. Berbeda dengan penambahan linier yang dieksekusi langsung ke gambar, blok kode ini berfokus pada pembuatan sebuah *Look-Up Table* (LUT) atau tabel pemetaan nilai piksel. Penggunaan matriks LUT sangat efisien secara komputasi karena program mengkalkulasi ulang formula eksponensial hanya untuk 256 kemungkinan nilai piksel (0 hingga 255) menggunakan perulangan *for*, alih-alih menghitungnya berjuta-juta kali untuk setiap piksel pada gambar. Formula di dalam perulangan tersebut menormalkan nilai piksel ke rentang desimal 0 hingga 1, mengangkatannya dengan nilai invers gamma (1.0 dibagi 2.5), dan mengalikannya kembali dengan 255. Secara teoretis, keluaran visual yang akan dihasilkan bila tabel LUT ini diaplikasikan pada citra adalah peningkatan kecerahan yang adaptif; area gelap dan bayangan akan diangkat (*boost*) kecerahannya secara signifikan sehingga detail subjek terlihat jelas, namun area yang sudah terang seperti sumber cahaya tidak akan dipaksakan menjadi putih murni, sehingga detail dan gradasi pada area bercahaya tetap terjaga dengan baik.

```
In [21]: clahe = cv2.createCLAHE(  
        clipLimit=4.5,  
        tileGridSize=(8, 8)  
    )  
  
    image_contrast = clahe.apply(image_photo_gray)
```

Blok kode tersebut mengimplementasikan teknik peningkatan kualitas citra tingkat lanjut yang dikenal sebagai *Contrast Limited Adaptive Histogram Equalization* (CLAHE) untuk memperbaiki kontras gambar secara lokal. Berbeda dengan ekualisasi histogram global yang sering kali memperburuk *noise* (bintik gangguan) yang menyebabkan area terang menjadi terlalu silau pada foto malam hari, metode CLAHE bekerja secara adaptif. Pada kode tersebut, program menginisialisasi objek CLAHE menggunakan fungsi `cv2.createCLAHE` dengan dua parameter krusial. Pertama, parameter `tileGridSize=(8, 8)` menginstruksikan algoritma untuk memecah citra menjadi blok-blok matriks kecil berukuran 8x8 piksel. Pemecahan ini memungkinkan proses kalkulasi pemerataan kontras dan distribusi histogram dilakukan secara spesifik pada masing-masing area lokal, bukan pada keseluruhan gambar sekaligus. Kedua, parameter `clipLimit=4.5` diterapkan sebagai ambang batas pemotongan puncak histogram. Nilai ini sangat penting untuk mencegah terjadinya amplifikasi *noise* berlebihan pada area gambar yang minim tekstur; jika ada puncak histogram lokal yang melampaui batas 4.5, maka nilai piksel yang berlebih tersebut akan dipotong dan didistribusikan secara merata ke keranjang histogram lainnya sebelum proses ekualisasi dilakukan.

Setelah parameter dikonfigurasi, perintah `.apply(image_photo_gray)` mengeksekusi algoritma CLAHE tersebut ke dalam citra berformat derajat keabuan (*grayscale*) dan menyimpannya pada variabel `image_contrast`.

```
In [22]: image_bright_contrast = clahe.apply(img_bri)
```

Blok kode ini merupakan penggabungan kontras dan pencahayaan.

```
In [23]: fig, axes = plt.subplots(2, 2, figsize=(14, 12))

axes[0, 0].imshow(image_photo_rgb)
axes[0, 0].set_title('Gambar Asli', fontsize=12)
axes[0, 0].axis('off')

axes[0, 1].imshow(image_photo_gray, cmap='gray')
axes[0, 1].set_title('Gambar Grayscale', fontsize=12)
axes[0, 1].axis('off')

axes[1, 0].imshow(img_bri, cmap='gray')
axes[1, 0].set_title('Gambar Gray yang Dipercerah', fontsize=12)
axes[1, 0].axis('off')

axes[1, 1].imshow(image_contrast, cmap='gray')
axes[1, 1].set_title('Gambar Gray yang Diperkontras', fontsize=12)
axes[1, 1].axis('off')

plt.suptitle('Perbaikan Gambar Backlight', fontsize=14, fontweight='bold')

plt.tight_layout()

plt.show()
```

Blok kode tersebut merupakan tahapan penyajian akhir dari rangkaian proses perbaikan kualitas citra, yang berfungsi untuk mengkompilasi dan menampilkan komparasi visual secara komprehensif. Menggunakan pustaka Matplotlib, program menginisialisasi sebuah kanvas berukuran 14x12 inci yang dibagi ke dalam tata letak *grid* matriks 2x2 melalui perintah `plt.subplots(2, 2)`. Pada masing-masing kuadran, program memproyeksikan tahapan transformasi citra yang telah dilakukan sebelumnya menggunakan fungsi `.imshow()`. Kuadran kiri atas (`axes[0, 0]`) diisi dengan "Gambar Asli" dalam format ruang warna RGB sebagai titik acuan komparasi. Kuadran kanan atas (`axes[0, 1]`) menampilkan citra hasil konversi derajat keabuan ("Gambar Grayscale"). Kuadran kiri bawah (`axes[1, 0]`) memproyeksikan citra hasil modifikasi kecerahan linier ("Gambar Gray yang Dipercerah"), dan kuadran kanan bawah (`axes[1, 1]`) menampilkan citra hasil ekualisasi kontras adaptif CLAHE ("Gambar Gray yang Diperkontras"). Untuk menjaga kebersihan visual pada laporan, fungsi `axis('off')` secara konsisten dipanggil pada setiap plot guna menghilangkan angka sumbu koordinat X dan Y. Keseluruhan kanvas ini kemudian diikat oleh satu judul utama "Perbaikan Gambar Backlight" menggunakan fungsi `plt.suptitle()`.



Gambar tersebut merupakan sebuah dasbor komparasi visual berukuran matriks 2x2 yang mendemonstrasikan efektivitas berbagai metode perbaikan kualitas citra digital, khususnya dalam menangani kasus degradasi visual akibat fenomena *backlight*. Pada kuadran kiri atas, "Gambar Asli" menampilkan citra sumber yang mengalami masalah *underexposed* parah pada area latar depan. Akibat dominasi sumber cahaya terang dari lampu di area latar belakang, sensor kamera menurunkan tingkat eksposur secara global, yang menyebabkan objek utama tidak mendapatkan iluminasi yang cukup sehingga menggelap dan menyerupai siluet tanpa detail. Tepat di sebelahnya pada kuadran kanan atas, "Gambar Grayscale" menampilkan representasi citra yang telah dikonversi ke dalam format derajat keabuan, berfungsi sebagai tahap prapemrosesan esensial untuk memfokuskan komputasi pada manipulasi intensitas cahaya (luminans) tanpa terdistraksi oleh kompleksitas saluran warna RGB.

Evaluasi inti dari *output* ini terletak pada perbandingan dua metode manipulasi intensitas piksel yang disajikan pada baris bawah. Pada kuadran kiri bawah, "Gambar Gray yang Dipercah" menampilkan hasil dari operasi peningkatan kecerahan piksel secara linier. Secara visual, metode ini terbukti kurang optimal; penambahan nilai kecerahan yang seragam pada seluruh matriks gambar memang membuat siluet subjek menjadi sedikit lebih terang, namun konsekuensinya area latar belakang yang aslinya sudah terang menjadi sangat silau. Selain itu, gambar secara keseluruhan kehilangan kontras dinamisnya sehingga tampak pudar, berkabut dan detail fitur wajah subjek tetap tidak dapat terekstraksi dengan baik.

Sebaliknya, pada kuadran kanan bawah, "Gambar Gray yang Diperkontras" menampilkan hasil perbaikan yang jauh lebih superior melalui pendekatan ekualisasi kontras lokal secara adaptif. Visualisasi ini membuktikan bahwa metode perbaikan kontras mampu mendistribusikan ulang rentang intensitas piksel secara cerdas dan spesifik pada blok-blok area tertentu. Hasilnya, detail tekstur pakaian dan fitur wajah subjek yang sebelumnya tersembunyi di balik bayangan gelap gulita berhasil direkonstruksi dan ditampilkan dengan ketajaman yang sangat baik. Signifikansi dari metode ini adalah kemampuannya dalam melakukan pemulihan detail visual pada area gelap secara agresif, namun tetap mampu menjaga integritas area terang pada latar belakang agar tidak mengalami distorsi kecerahan berlebih. Oleh karena itu, dapat disimpulkan

bahwa metode peningkatan kontras adaptif merupakan solusi yang paling efektif dan ideal untuk mengoreksi citra yang terdampak fenomena *backlight*.

```
In [24]: plt.figure(figsize=(7, 6))
plt.imshow(image_bright_contrast, cmap='gray')
plt.title('Gambar Gray yang Dipercerah dan Diperkontras', fontsize=12, fontweight='bold')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Blok kode tersebut merupakan tahapan finalisasi visualisasi yang bertujuan untuk menyajikan citra hasil kombinasi dari dua metode perbaikan kualitas, yaitu peningkatan kecerahan dan penajaman kontras (*contrast*) secara bersamaan. Secara prosedural, program pertama-tama mengalokasikan ruang kanvas visual menggunakan fungsi `plt.figure(figsize=(7, 6))` untuk menetapkan dimensi bingkai gambar sebesar 7x6 inci. Selanjutnya, perintah `plt.imshow` dieksekusi untuk memproyeksikan matriks data piksel yang tersimpan dalam variabel `image_bright_contrast`. Penggunaan parameter `cmap='gray'` pada fungsi ini bersifat wajib untuk menginstruksikan sistem agar merender gambar menggunakan peta warna derajat *grayscale* sehingga representasi intensitas cahayanya tidak terdistorsi. Guna memberikan konteks informasi pada grafik keluaran, program menyematkan teks menggunakan fungsi `plt.title` dengan ukuran font 12. Untuk memperoleh estetika tampilan citra yang bersih menyerupai foto asli, perintah `plt.axis('off')` dipanggil untuk menghilangkan elemen garis ukur sumbu koordinat X dan Y. Rangkaian tahapan ini kemudian diakhiri dengan eksekusi `plt.tight_layout()` untuk merapikan margin tata letak secara otomatis, sebelum akhirnya keseluruhan grafik dirender dan ditampilkan secara utuh ke layar antarmuka melalui perintah `plt.show()`.

Gambar Gray yang Dipercerah dan Diperkontras



Gambar diatas menyajikan visualisasi final dari penerapan kombinasi dua teknik perbaikan kualitas citra secara simultan, yaitu peningkatan kecerahan dan penajaman kontras adaptif. Secara analitis, citra ini mendemonstrasikan solusi yang paling komprehensif dalam mengatasi degradasi visual akibat fenomena *backlight* ekstrem pada foto sumber. Melalui pendekatan hibrida ini, operasi peningkatan kecerahan berfungsi untuk mengangkat rentang nilai intensitas piksel secara keseluruhan, khususnya untuk

mengeliminasi area *underexposed* pada latar depan. Di saat yang bersamaan, penerapan algoritma ekualisasi kontras (seperti CLAHE) bekerja secara lokal untuk mendistribusikan ulang histogram piksel, sehingga mencegah citra menjadi pudar akibat penambahan cahaya tersebut. Hasil akhirnya menunjukkan *recovery* informasi visual yang sangat signifikan; fitur wajah dan detail tekstur pakaian pada objek utama yang sebelumnya tersembunyi di balik siluet gelap kini berhasil direkonstruksi dan ditampilkan dengan tingkat kejelasan yang optimal. Selain itu, elemen di sekitarnya seperti tekstur dedaunan dan pola blok jalan juga menjadi jauh lebih tajam dan terdefinisi dengan baik, membuktikan bahwa integrasi kedua metode ini mampu menghasilkan citra dengan visibilitas subjek yang maksimal sekaligus mempertahankan dinamika rentang cahaya yang seimbang.

BAB IV

PENUTUP

Praktikum ini telah berhasil merealisasikan penerapan algoritma pengolahan citra digital untuk kebutuhan segmentasi warna dan manipulasi pencahayaan secara komputasional. Berdasarkan hasil analisis dan pembahasan, teknik pemisahan kanal warna RGB yang dikombinasikan dengan operasi logika *thresholding* terbukti efektif dalam mengekstraksi objek teks berdasarkan pigmen warnanya secara spesifik. Penggunaan nilai ambang batas yang dikonfigurasi secara empiris memainkan peran krusial dalam mengeliminasi *noise* visual yang ditimbulkan oleh ketidakmerataan cahaya pada latar belakang kertas. Selain itu, evaluasi menggunakan representasi grafik histogram telah memberikan validasi matematis yang akurat. Transformasi bentuk histogram dari yang awalnya bersifat kontinu pada citra asli menjadi sangat diskrit pada citra keluaran membuktikan bahwa proses pemetaan matriks, *masking* biner, dan klasifikasi piksel target telah dieksekusi dengan tingkat presisi yang tinggi sesuai dengan logika algoritma yang dirancang.

Lebih lanjut, eksperimen terhadap metode peningkatan kualitas citra pada kasus foto dengan kondisi *backlight* menunjukkan hasil komparasi yang sangat signifikan. Pendekatan modifikasi kecerahan secara linier dinilai kurang optimal karena rentan memicu distorsi *over-exposure*, yang mengakibatkan hilangnya detail gambar dan membuat citra tampak pudar. Sebaliknya, penerapan metode *Contrast Limited Adaptive Histogram Equalization* (CLAHE) terbukti jauh lebih superior. Algoritma ini mampu beroperasi secara adaptif untuk mendistribusikan ulang intensitas piksel pada blok-blok matriks lokal. Hasilnya, CLAHE berhasil *recovery* informasi visual dan detail tekstur pada area siluet yang mengalami *underexposed*, tanpa mengorbankan atau merusak kualitas piksel pada area latar belakang yang sudah terpapar cahaya dengan baik.

DAFTAR PUSTAKA

- [1] Nasution, S., Supiyandi, et al. (2025). "Penerapan Metode Segmentasi Warna untuk Deteksi Objek Berbasis Warna pada Citra Digital," *Jurnal Teknik Informatika dan Terapan*, Vol. 3, No. 4, hal. 14-16.
- [2] Hidayat, J., & Fitriani, A. (2025). "Perbandingan Metode Power Law dengan Contrast Limited Adaptive Histogram Equalization (CLAHE) pada Perbaikan Kualitas Citra," *Jurnal Teknik Elektro*, Vol. 8.
- [3] "Analysis of Combined Contrast Limited Adaptive Histogram Equalization (CLAHE) and Median Filter Methods for Enhancement of CCTV Screenshot Image Quality," *Journal of Informatics and Telecommunication Engineering*, Vol. 3, No. 1, hal. 49-56.